

# Python for AI & Machine Learning

## Class 11 – Introduction to Object-Oriented Programming (OOP)

**Trainer:** Surya Dekshith Mallikarjuna

**Course:** Python for AI & Machine Learning

**Duration:** 2 Hours (Theory + Practical)

---

### Learning Objectives

After completing this session, students will be able to:

- Understand the concept of Object-Oriented Programming (OOP).
  - Differentiate between Procedural Programming and OOP.
  - Understand Classes and Objects.
  - Create classes and objects in Python.
  - Understand Attributes and Methods.
  - Learn the purpose of the `self` keyword.
  - Understand Constructors ( `__init__()` ).
  - Build simple real-world applications using OOP.
  - Understand why OOP is essential for AI and Machine Learning.
- 

### Introduction

Until now, we have written programs using functions and variables. As applications grow larger, managing code becomes difficult. To solve this problem, Python provides **Object-Oriented Programming (OOP)**.

Object-Oriented Programming is a programming paradigm that organizes code into reusable objects, making software easier to develop, maintain, and expand.

OOP is widely used in:

- Artificial Intelligence
- Machine Learning
- Data Science
- Web Development
- Mobile App Development
- Game Development
- Enterprise Software

Most modern Python libraries used in AI and ML, such as NumPy, Pandas, Scikit-learn, TensorFlow, and PyTorch, are designed using Object-Oriented Programming principles.

---

## Procedural Programming vs Object-Oriented Programming

| Procedural Programming                   | Object-Oriented Programming             |
|------------------------------------------|-----------------------------------------|
| Uses functions                           | Uses classes and objects                |
| Difficult to maintain large applications | Easier to maintain                      |
| Less reusable                            | Highly reusable                         |
| Suitable for small programs              | Suitable for large applications         |
| Data and functions are separate          | Data and functions are grouped together |

---

## Real-Life Example

Consider a **Car**.

A car has:

### Properties

- Brand
- Model
- Color
- Engine Number
- Speed

### Behaviors

- Start
- Stop
- Accelerate
- Brake

The **Car** is called a **Class**.

A specific car like:

- Hyundai Creta
- Tata Nexon
- Maruti Swift

is called an **Object**.

A class acts as a blueprint, while objects are the real-world instances created from that blueprint.

---

## What is a Class?

A **Class** is a blueprint used to create objects.

A class defines:

- Variables (Attributes)
- Functions (Methods)

### Syntax

```
class Student:  
    pass
```

The `pass` keyword indicates that the class is currently empty.

---

## What is an Object?

An object is an instance of a class.

Once a class is created, multiple objects can be created from it.

### Example

```
class Student:  
    pass  
  
student1 = Student()  
  
print(student1)
```

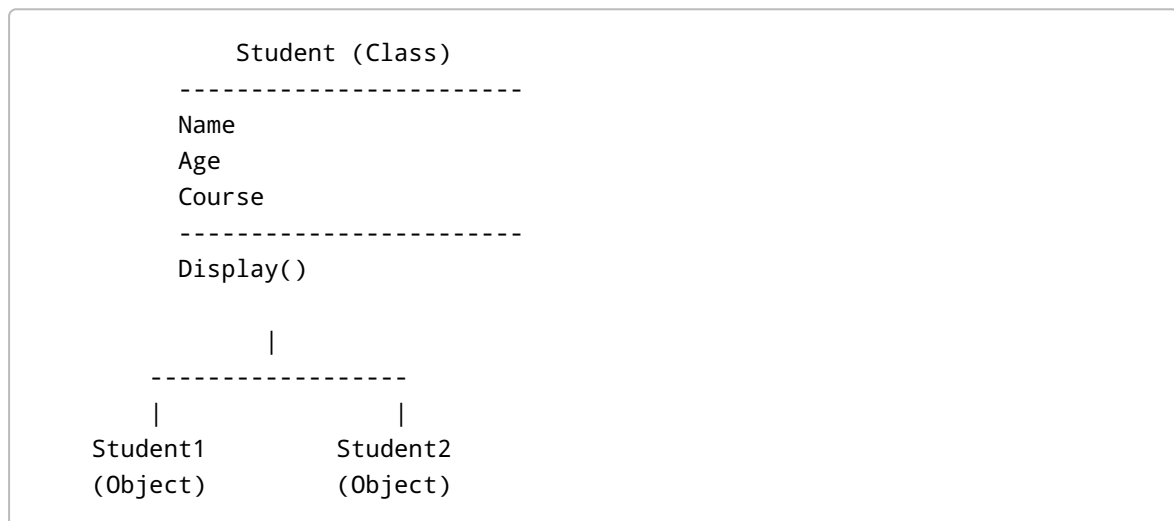
### Output

```
<__main__.Student object at 0x...>
```

Python creates the object in memory.

---

# Class Diagram



## Attributes

Attributes are variables defined inside a class.

### Example

```
class Student:

    name = "Rahul"
    age = 22

student = Student()

print(student.name)
print(student.age)
```

### Output

```
Rahul
22
```

## Methods

Methods are functions defined inside a class.

They describe the actions an object can perform.

### Example

```
class Student:

    def welcome(self):
        print("Welcome to AI & ML Class")

student = Student()

student.welcome()
```

### Output

```
Welcome to AI & ML Class
```

---

## Understanding the self Keyword

The `self` keyword represents the current object.

Whenever a method is called using an object, Python automatically passes that object as the first argument.

### Example

```
class Student:

    def display(self):
        print("This method belongs to the current object.")

student = Student()

student.display()
```

---

## Constructor

A constructor is a special method that executes automatically whenever an object is created.

In Python, the constructor is written using:

```
__init__()
```

## Example

```
class Student:

    def __init__(self):
        print("Constructor Executed")

student = Student()
```

## Output

```
Constructor Executed
```

---

# Constructor with Parameters

```
class Student:

    def __init__(self, name, age):

        self.name = name
        self.age = age

student = Student("Rahul", 22)

print(student.name)
print(student.age)
```

## Output

```
Rahul
22
```

---

# Creating Multiple Objects

```
class Student:

    def __init__(self, name, course):

        self.name = name
        self.course = course
```

```
student1 = Student("Rahul", "AI & ML")
student2 = Student("Priya", "Python")

print(student1.name)
print(student2.name)
```

### Output

```
Rahul
Priya
```

Each object stores its own data independently.

---

## AI Example 1 – AI Student Management

```
class AIStudent:

    def __init__(self, name, course):

        self.name = name
        self.course = course

    def display(self):

        print("Student Name :", self.name)
        print("Course :", self.course)

student = AIStudent("Surya", "AI & ML")

student.display()
```

## AI Example 2 – Machine Learning Model

```
class MLModel:

    def __init__(self, model_name):

        self.model_name = model_name

    def train(self):

        print(self.model_name, "Model Training Started")
```

```
model = MLModel("Linear Regression")  
  
model.train()
```

### Output

```
Linear Regression Model Training Started
```

---

## AI Example 3 – Chatbot

```
class ChatBot:  
  
    def __init__(self, name):  
  
        self.name = name  
  
    def greet(self):  
  
        print("Hello! I am", self.name)  
  
bot = ChatBot("Digix AI")  
  
bot.greet()
```

### Output

```
Hello! I am Digix AI
```

---

## Why is OOP Important in AI & Machine Learning?

Almost every AI library is built using classes and objects.

Examples include:

- Creating Machine Learning models
- Loading datasets
- Building Neural Networks
- Image Processing
- Natural Language Processing
- Chatbots

- Deep Learning

For example:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)

prediction = model.predict(X_test)
```

Here:

- `LinearRegression` is a class.
- `model` is an object.
- `fit()` and `predict()` are methods.

Understanding OOP makes it much easier to work with AI libraries.

---

## Advantages of OOP

- Code reusability
  - Better organization
  - Easier debugging
  - Improved maintenance
  - Scalable applications
  - Better security
  - Faster development
  - Real-world modeling
- 

## Summary

In this class, we learned:

- Introduction to OOP
- Difference between Procedural Programming and OOP
- Classes
- Objects
- Attributes
- Methods
- The `self` keyword
- Constructors (`__init__()`)
- Creating multiple objects
- Practical AI-based examples
- Importance of OOP in AI and Machine Learning

---

# Practical Exercises

## Exercise 1

Create an **Employee** class.

Store:

- Employee ID
- Employee Name
- Department

Create three employee objects and display their details.

---

## Exercise 2

Create a **Book** class.

Store:

- Book Name
- Author
- Price

Create five book objects.

Display all book information.

---

## Exercise 3

Create a **Laptop** class.

Store:

- Brand
- RAM
- Processor

Create two laptop objects.

Display the specifications.

---

## Exercise 4

Create an **AIModel** class.

Store:

- Model Name
- Algorithm
- Accuracy

Create two AI models and display their details.

---

## Assignment

Develop a **Hospital Management System** using Object-Oriented Programming.

Create a `Patient` class with the following attributes:

- Patient ID
- Patient Name
- Age
- Disease
- Doctor Name

Create the following methods:

- `admit()`
- `display_details()`

Create details for at least **five patients** and display all patient information.

---

## Next Class Preview

In the next session, we will learn:

- Instance Variables
- Class Variables
- Types of Methods
- Instance Methods
- Class Methods
- Static Methods
- Advanced Constructor Concepts
- Object Interaction
- More AI-oriented OOP examples